

Generative AI and LLM-based Agents

Fundamentals of Digitalization and Digital Trends — Lecture 01

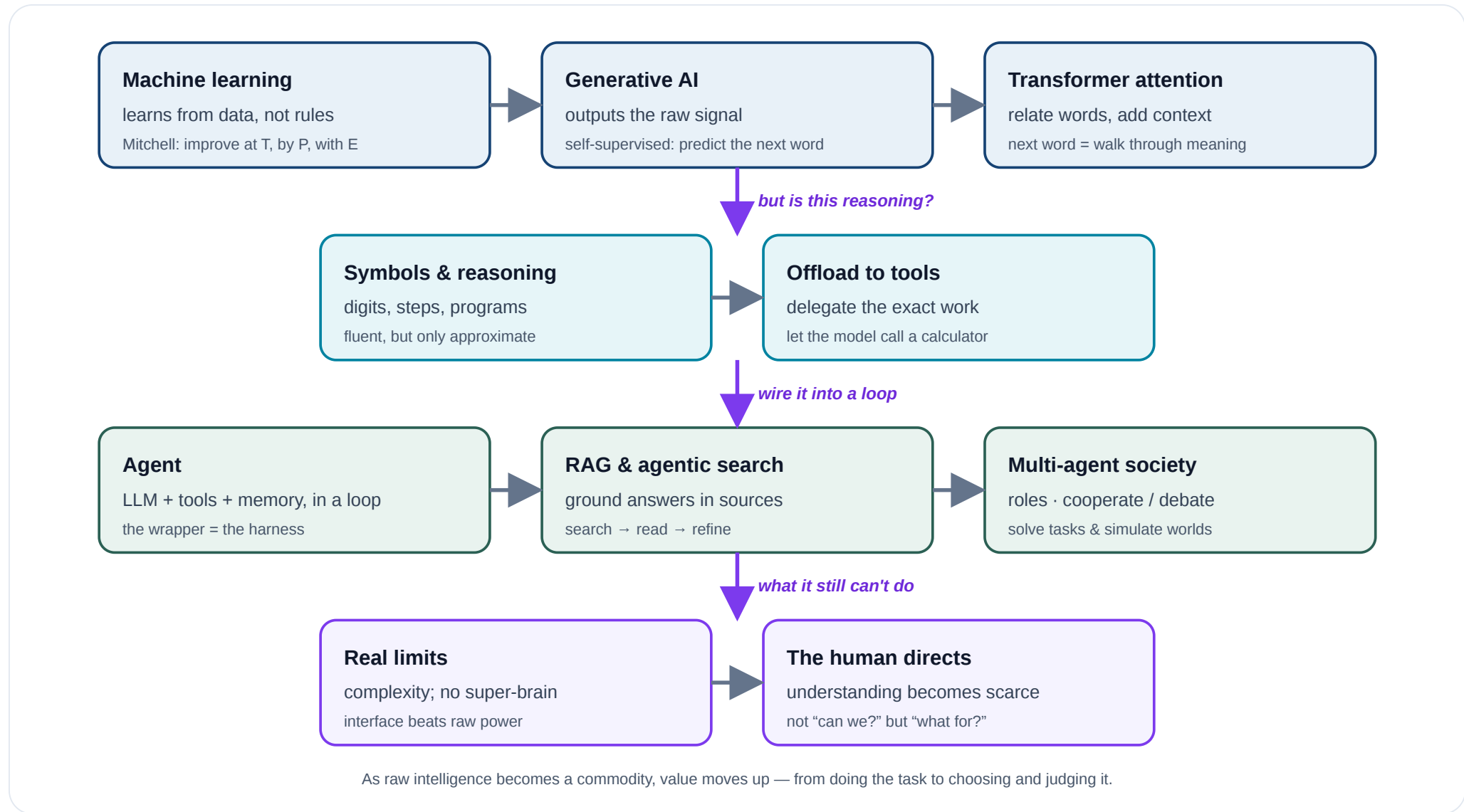
Prof. Dr. Michael Granitzer

14 June 2026

Where we are going

The question that frames this lecture: generative AI is making intelligence a commodity you can *buy by the token* — **how** is that capability built, **where** does it reach its limits, and **what** does that imply for the future division of labour between humans and machines?

Agendas



The technology is only half the lecture; the other half is the question it forces on us — **follow the downward arrows.**

1 — From Machine Learning to Generative AI

Guiding question: What does it mean for a machine to *learn*, and what is new about *generating* text and images?

What is machine learning?

A classic, precise definition:

"A computer program is said to **learn** from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in T, as measured by P, improves with experience E." — Mitchell (1997) (Mitchell 1997)

We do **not** write the rules by hand. We specify a task, a way to measure success, and let the program improve from experience, usually given in some form of data.

The main kinds of learning



Mitchell (1997): a program learns from experience **E w.r.t. tasks **T** and measure **P**, if **P** on **T** improves with **E**.**

T — task

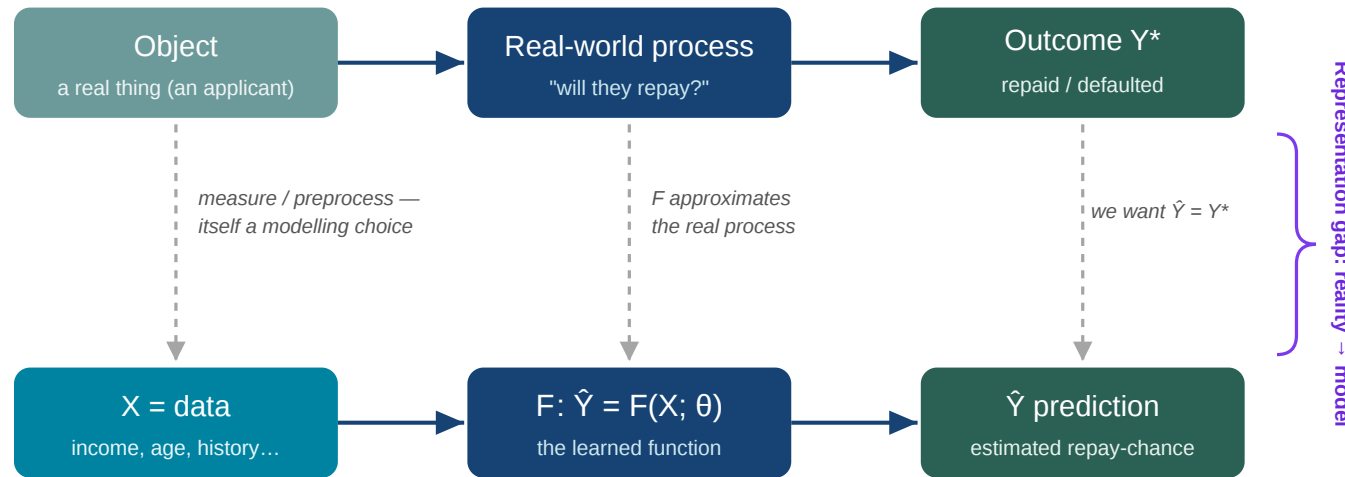
P — performance

E — experience



From world to model

REAL WORLD — the process we want to approximate



MODEL — an approximation built from data

Both the data X and the function F are *our chosen models* of a messy reality.

A model is an **approximation of a real-world process**, written $\hat{Y} = F(X; \theta)$:

- X — the **data**: our chosen measurement of a real object (*preprocessing is already a modelling decision*)
- F — the **function** standing in for the real process; θ are its tunable **parameters** ("knobs")
- $\hat{Y}$ — the **prediction**; we want it close to the true outcome Y^*
-
- For unsupervised problems Y^* is unknown and not available in the loss function

Abstracting a messy reality into X and F is the **representation gap** — even *what we choose to measure* is already a modelling decision.

Learning = estimating θ from a sample



We never see the whole world — only a **sample** (x_i, y_i) drawn from a population; that distance between sample and population is the **estimation gap**. Training **tunes the knobs** to fit the sample:

$$\theta^* = \operatorname{argmin}_{\theta} \sum_i L(F(x_i; \theta), y_i)$$

The **loss** $L(\hat{y}, y)$ measures *how wrong* the model is; the **optimizer** turns the knobs to reduce it.

Learning = estimating consequences of the gaps: the model can be **confidently wrong** on cases it never saw, and a **skewed sample** → a **biased model**.

Predictive learning: the same picture, filled in

Here Y is a **label** we want to predict, and the **loss** scores *how wrong* each guess is — same skeleton, only X , the label, and the loss change:

Task	X (input)	F	Y (label)	Loss — "how wrong?"
Credit	income, age, history	classifier	repay? yes / no	a cost per wrong call — approving a defaulter hurts most
Spam	the email text	classifier	spam / not spam	count the misfiled emails
Vision	the pixels of a photo	classifier	cat / dog	share of photos mislabelled

The optimizer then tunes θ to make that loss as small as possible.

The leap to generative AI is just a change in **what Y is** plus

1. a new kind of training algorithm: Transformers
2. massive amounts of data and corresponding infrastructure

Discriminative vs. generative

Same skeleton $\hat{Y} = F(X; \theta)$ — the difference is **what Y is**:

Discriminative — *Y is a label*

A verdict about the input.

- spam / not spam
- credit risk score
- cat vs. dog

Generative — *Y is the raw signal*

New content of the same kind as the data.

- write a paragraph
- draw an image
- compose audio

The leap of generative AI: it produces the **raw signal** — text, pixels, sound — not just a verdict about it.

Generative learning in the same picture

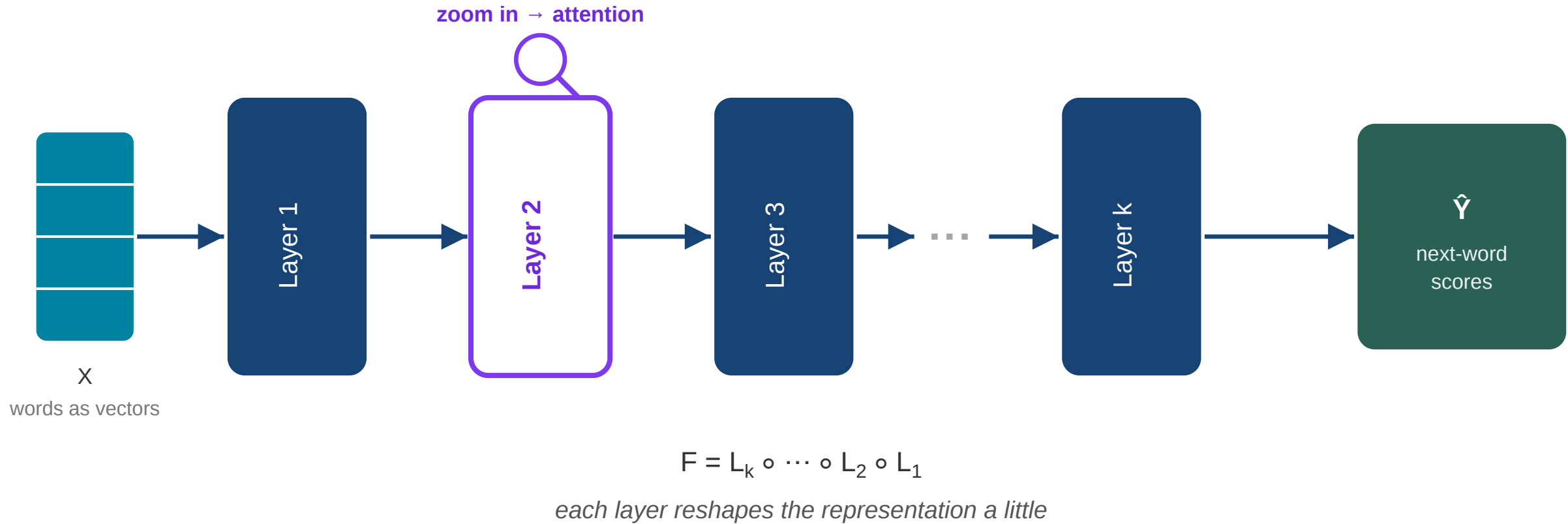
A language model is just our skeleton with a special choice of X and Y :

- X = **the words so far**, and Y = **the next word** — so $\hat{Y} = F(X; \theta)$
- **self-supervised**: every sentence is its own training data — the next word *is* the label, so no human labelling is needed
- loss = "did it predict the actual next word?"; the optimizer tunes θ over **billions** of sentences

Generate by repeating: predict the next word, append it, feed it back in. Consequently, language understanding seems to be a **supervised** learning problem.

So *what is inside F* that turns "the words so far" into a good next word?

Inside F: a stack of layers



F is built from many simple **layers** stacked in sequence — $F = L_k \circ \dots \circ L_1$. Each layer takes the current representation and **reshapes it** a little. Zoom into one layer and you find the transformer's key idea: **attention**.

Attention: what one layer does

Zoom into one layer. For a target word it:

1. reads **all** the input words — each is a **vector** (a point in meaning-space);
2. **scores** how related every word is to the target — one number per word;
3. **attends**: takes a weighted mix of the word-vectors, by those scores;
4. outputs a **new vector** for the target — its context-aware meaning.

So the shape is **vectors in** → **scores** → **a new vector out**.
Here "bank" attends most to "river", so its output vector is pulled toward the *riverside* region of the map.

1 · Input — each word a vector

The	(0.1 , 0.2)
river	(0.9 , 0.1)
flows	(0.4 , 0.3)
to the	(0.2 , 0.2)
bank	(0.5 , 0.5)

← target

2 · Attention — a score for every word

how much should "bank" listen to each?

The		0.05
river		0.55
flows		0.20
to the		0.10
bank		0.10

3 · Output — a new vector for "bank"

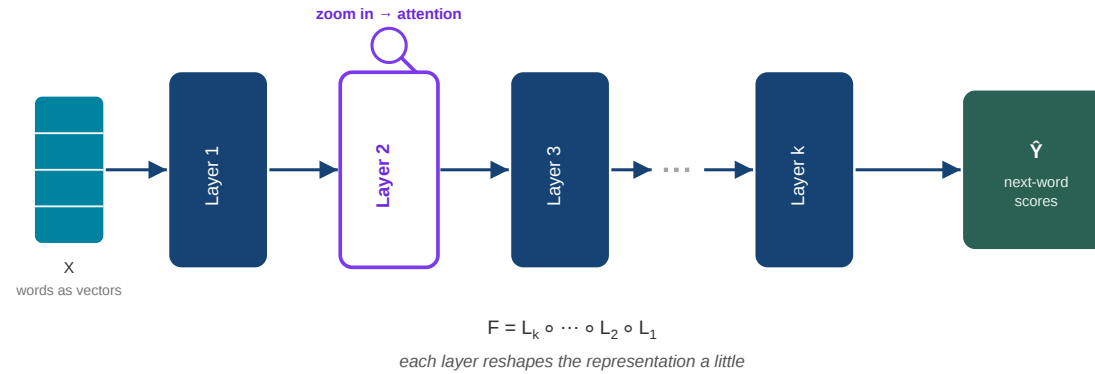
weighted mix of the inputs by their scores



bank' = (0.8 , 0.15)

pulled toward "river" → means riverside

Layers move points across a map of meaning



Picture every word as a **point** on a **map of meaning**, where related words sit close. **One layer = one step** that moves each point across the map.



Each layer is a step across the map



Each transformer layer **nudges** every word's point across the map, using context. "Bank" starts ambiguous, between the regions; layer by layer the context *"deposit money at the ..."* pushes it into the finance region.

After the last layer, we land in a region — and the **nearest words there** (money, account) are the model's prediction.

Predicting the next word is a **guided walk through a space of meaning** — context steers

the path, and where you land is what you say.

2 — Computing with Symbols: reasoning or repetition?

Guiding question: Is "predict the next symbol" real reasoning, or a *stochastic parrot* (Bender et al. 2021) repeating patterns?

Two tasks — hard to just *know*

A · Arithmetic

Compute $2840 \div 6$ — *exactly*.

B · A process

List **every step** to register at the university and enrol in *this* course.

Neither answer can be **recalled** — you must **work through it**, one correct step at a time.

Both are solved by *symbolic reasoning*

A · Symbolic calculation — $2840 \div 6$

- $6 \times 4 = 24$, so $28 - 24 = 4$
- bring down 4 $\rightarrow 44$; $6 \times 7 = 42 \rightarrow 2$
- bring down 0 $\rightarrow 20$; $6 \times 3 = 18 \rightarrow 2$
- $\Rightarrow$ **473, remainder 2**

B · A process description

1. apply / register online
2. upload documents $\rightarrow$ matriculation
3. activate the campus account
4. enrol in *this course* in the system
5. confirm and pay the fee

In both we **manipulate symbols by fixed rules**, step by step — that is **symbolic reasoning**.

What *is* reasoning?

Reasoning = start from **axioms** (accepted facts) and apply **rules of inference** to derive new, *guaranteed* conclusions.

- **Axioms:** "All humans are mortal." · "Socrates is a human."
- **Rule:** if *all A are B* and *x is an A*, then *x is B*.
- **⇒ Conclusion:** "Socrates is mortal."

Three properties: it is **provable** (every step is shown), **correct** (if the axioms and rules are), but can be **costly** (some chains are very hard to find).

Reasoning, more formally

A **formal system** = **axioms** + **inference rules**. A **proof** is a finite chain of statements, each one an axiom or derived from earlier ones by a rule.

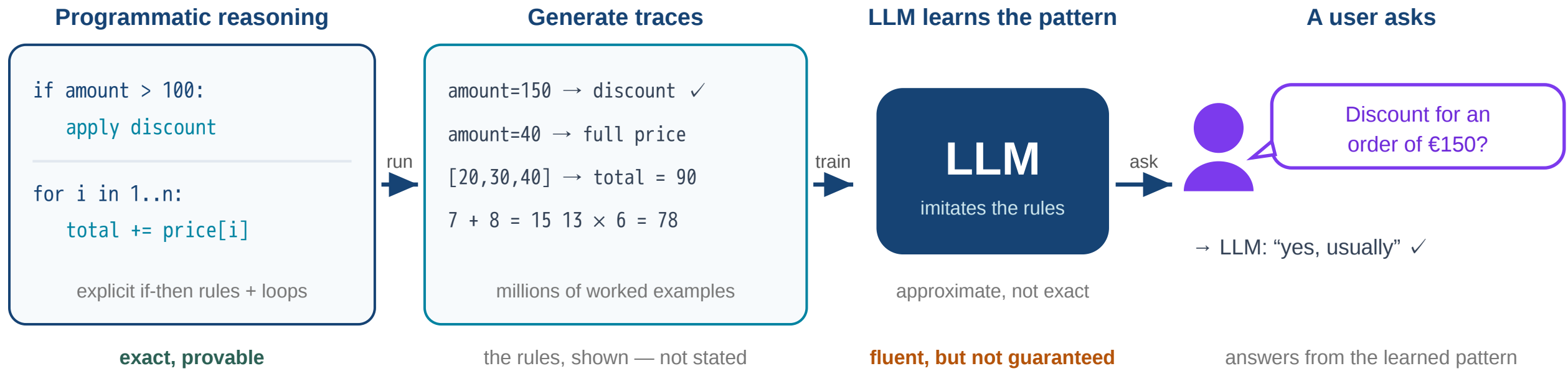
Socrates as a derivation — the rule is *modus ponens* (from A and "A $\rightarrow$ B", infer B):

$\forall x.$	Human(x) $\rightarrow$ Mortal(x)	(general rule / axiom)
	Human(Socrates)	(fact)
$\therefore$	Mortal(Socrates)	(by modus ponens)

- **Sound** — true axioms + valid rules $\Rightarrow$ every statement derived is *true*.
- **Costly** — finding a proof can be intractable: propositional satisfiability is already **NP-complete**; full first-order logic is **undecidable**.

Do LLMs reason?

Rules and programs **generate traces**; by predicting the next step, the LLM **imitates the pattern** — then answers new questions:



But they do **not** run exact algorithms like a calculator. Anthropic's interpretability work finds a model adds numbers with **approximate, parallel heuristics** — not the carry algorithm — and then cannot faithfully report how it did it (Lindsey et al. 2025).

So it is **approximate reasoning**: often right and fluent, but **not guaranteed** — and oddly **expensive and unreliable** for exact work like adding large numbers.

We offload exact work to a tool

Reasoning most often requires high effort.

So what do humans actually do? We **offload** exact symbolic work to a tool.

Brain

judgement, goals

Calculator

exact arithmetic

Result

fast & correct

The skill that matters is not doing every step in your head — it is knowing **what to compute** and **delegating** it to something reliable.

Symbols are not just digits

Arithmetic is symbolic — but so are **processes**, **procedures**, and **programs**.

Arithmetic

$473 \times 6 \dots$

Process

step 1 → step 2 → step 3

Program

code that runs

The **same next-symbol machine** that predicts digits can predict the next **step of a workflow** or the next **line of code**.

So an LLM can do more than call one tool — it can **write and drive a whole process**: a **controller of a workflow** that invokes many tools in sequence. That is an agent.

3 — LLM-based Agents

Guiding question: How does a text predictor become a system that *acts* — searching, computing, remembering?

 *Required reading: Guo et al. (2024), the multi-agent survey — this module maps directly onto it.*

Agent = LLM + Tools + Memory



LLM

reason & plan

Tools

act in the world

Memory

carry context

The scaffolding that wraps the model — the **loop**, the **tools**, the **memory** and context management — is called the **agent harness**. The model does the reasoning; the harness

is everything else that turns a raw LLM into a working agent.

Giving the model knowledge: RAG

Retrieval-Augmented Generation — look things up *before* answering.

1 · Build the index — offline



Vector search matches **meaning**, not exact words — it finds passages about the same idea even when they share no keywords (synonyms, paraphrase, other languages).

A pure model can **hallucinate**. RAG curbs this by **grounding** the answer in retrieved sources it can cite.

Why vector search? keywords vs. meaning

Sparse index



Dense index



Sparse / keyword — an inverted file: the query word must *literally appear*. Fast, exact, but blind to wording.

Dense / vector — encode query and chunks into vectors; retrieve the **nearest** ones. Matches *meaning*, so it survives synonyms and paraphrase.

From retrieval to agentic search

Plain RAG — *one shot*

retrieve once → answer.

Good for direct, single-fact questions.

Agentic search — *a loop*

search → read → refine the query →
search again → synthesise.

The agent decides *what to look up next*,
based on what it just learned.

Agentic retrieval: the workflow

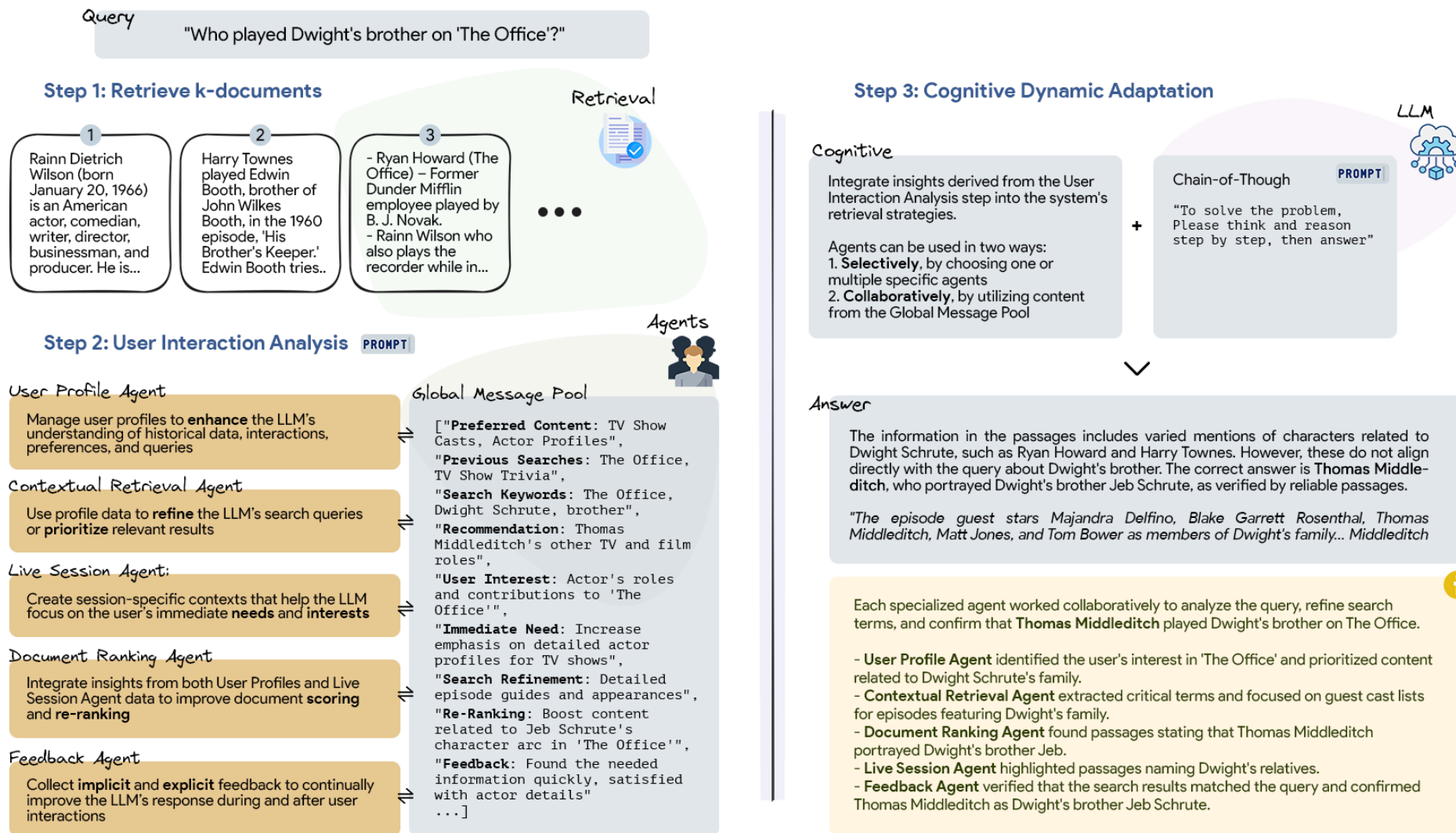


Search becomes an **investigation**, not a single lookup:

- **plan** the next sub-question
- **retrieve** with search + tools
- **check** the evidence so far
- **refine** and search again
- **answer** with provenance, once it is enough

Retrieval inside a control loop — the agent decides *what to look up next*, and tools + memory feed every step.

PersonaRAG — retrieval with user-centric agents



Plain RAG retrieves for a *query* — not for a *user's* intent and history.

PersonaRAG wraps retrieval in a small society of **user-centric agents**:

- user-profile agent
- contextual-retrieval agent
- live-session agent
- document-ranking agent
- feedback agent

The output is a **personalised evidence set**, not just a ranked list — **+10%** over vanilla RAG on WebQuestions (Zerhoudi & Granitzer 2024).

Source: Zerhoudi & Granitzer 2024

Many agents, not one



Multi-agent system can be organised along a few design choices (Guo et al. 2024) :

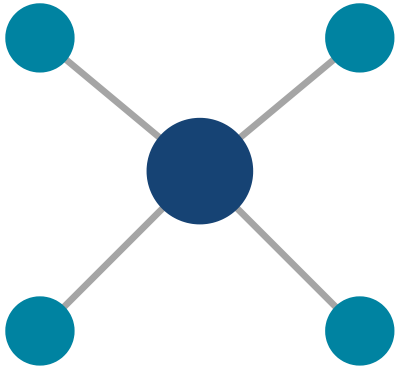
Profiling — give each agent a *role*: planner, researcher, coder, critic.

Communication — how they talk: *cooperate, debate, or compete*.

Capability — agents improve from *feedback* and shared *memory* → collective intelligence.

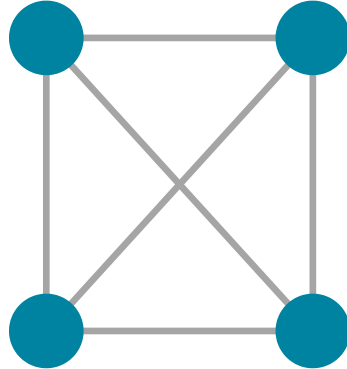
How agents coordinate

Centralized



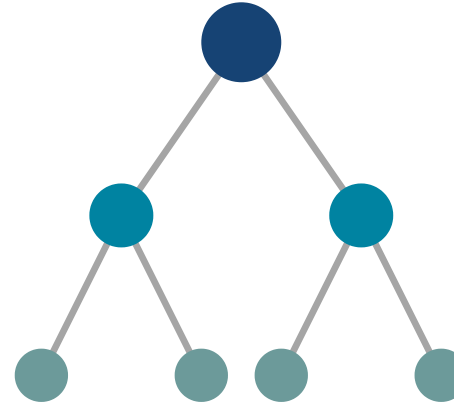
one coordinator

Decentralized



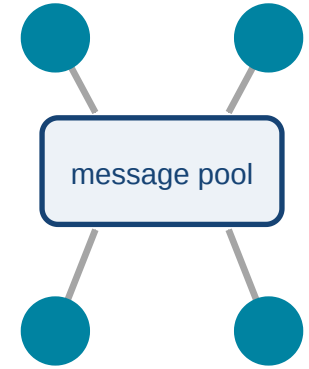
peer to peer

Layered



hierarchy

Shared pool



shared blackboard

Familiar from human organisations: a manager, a flat team, a hierarchy, or a shared workspace.

Cooperation — agents build on each other toward a shared goal.

Debate — agents argue opposing views; the clash *surfaces errors* and improves the answer.

Simulating whole worlds

Beyond *solving* tasks, multi-agent LLMs are used to **simulate** them (Guo et al. 2024):

Societies

opinion & social-media dynamics

Economies

markets, trading, policy

Populations

epidemics, mobility

Give agents personas, let them interact, and study the **emergent** behaviour — a new instrument for the social and economic sciences. (*Example: Park et al.'s "Smallville" — 25 agents in a sandbox town that plan their days, gossip, and organise a party on their own (Park et al. 2023).*)

Capability comes from **LLM + tools + memory + coordination** — and the same machinery becomes a *laboratory* for studying human systems.

4 — Limits and Human–AI Collaboration

Guiding question: What can AI *not* do for us — and what does that leave for humans?

A better interface beats raw power

In *freestyle* (centaur) chess, teams of **amateurs with laptops and a good process** beat both grandmasters and supercomputers (Kasparov 2017).

Kasparov's lesson: *weak human + machine + better process > strong human + machine + worse process, and > machine alone.*

The decisive variable was not horsepower — it was the **interface** between human and machine.

AI does not repeal hard problems

Some problems are hard for *anyone* — human or machine — because of their **computational complexity**.

Even **approximate** Bayesian inference is **NP-hard** (Dagum & Luby 1993). Scaling a model bigger does not dissolve an NP-hard problem.

More compute makes many things faster. It does **not** abolish the wall that complexity theory puts in front of us.

There is no single super-brain

Look at human intelligence: it is **not one giant brain**. It is a **society** of brains — specialised, cooperating, correcting each other.

Unlikely: one monolithic super-intelligence that does everything.

Likely: a **society of agents and humans** — specialised, networked, cooperating (Guo et al. 2024).

...but that society is hard to build

The same survey is candid about the **open challenges** (Guo et al. 2024) — each a reason humans stay in the loop:

Errors compound — one agent's hallucination propagates through the group.

Evaluation — what does "good" even *mean* for a whole society of agents?

Collective intelligence — getting agents to genuinely *learn together* is unsolved.

Cost — many agents = many model calls = a lot of compute.

So where does that leave the human?

In Andrej Karpathy's recent framing: **intelligence has become computable** — you can buy it by the token.

What becomes scarce is **understanding** — the high-level grasp to *guide* intelligence toward the right ends.

The question shifts from "**can we do it?**" to "**what *should* we do?**" — the **goal**, not the skill.

What to take away — and to build

As intelligence becomes a commodity, your edge is **understanding and direction**: asking the right questions, setting the right goals, and judging the answers.

For an interdisciplinary world: the technical "how" is increasingly automatable. The "why", the "should", and the "what for" are yours — and they decide whether the technology serves us.

Summary

Machine Learning → Generative AI → Symbols & Tools → Agents → a society of minds
→ the human as the one who *understands and directs*.

Discussion

- Where in **your** field is the bottleneck already shifting from *execution* to *understanding*?
- Which tasks would you happily delegate to an agent — and which would you never?
- If the future is a *society of agents and humans*, who is **accountable** when it errs?

Image Credits & References

Lindsey, Jack and Gurnee, Wes and Ameisen, Emmanuel and Templeton, Adly and Batson, Joshua and others (2025). On the Biology of a Large Language Model. <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>

Bender, Emily M. and Gebru, Timnit and McMillan-Major, Angelina and Shmitchell, Shmargaret (2021). On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*.

Dagum, Paul and Luby, Michael (1993). Approximating Probabilistic Inference in Bayesian Belief Networks Is NP-hard. *Artificial Intelligence*.

Guo, Taicheng and Chen, Xiuying and Wang, Yaqi and Chang, Ruidi and Pei, Shichao and Chawla, Nitesh V. and Wiest, Olaf and Zhang, Xiangliang (2024). Large Language Model Based Multi-Agents: A Survey of Progress and Challenges. *arXiv preprint arXiv:2402.01680*. <https://arxiv.org/abs/2402.01680>

Kasparov, Garry (2017). Deep Thinking: Where Machine Intelligence Ends and Human Creativity Begins. *PublicAffairs*.

Mitchell, Tom M. (1997). Machine Learning. *McGraw-Hill*.

Park, Joon Sung and O'Brien, Joseph C. and Cai, Carrie J. and Morris, Meredith Ringel and Liang, Percy and Bernstein, Michael S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.

Vaswani, Ashish and Shazeer, Noam and Parmar, Niki and Uszkoreit, Jakob and Jones, Llion and Gomez, Aidan N. and Kaiser, Lukasz and Polosukhin, Illia (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/1706.03762>

Zerhoudi, Saber and Granitzer, Michael (2024). PersonaRAG: Enhancing Retrieval-Augmented Generation Systems with User-Centric Agents. *arXiv preprint arXiv:2407.09394*. <https://arxiv.org/abs/2407.09394>